

How to Build and Run an AI Safety Pilot Study

A step-by-step guide using the Aegis research program as a worked example

Plain-English companion for non-specialists

Is the whole thing called a pilot?

Yes - calling the early Aegis work a pilot is reasonable, especially because the purpose is to learn whether the experimental method, prompts, scoring, controls, and formal specification behave as intended before making stronger claims. A more precise name for the whole sequence is a pilot research program or experimental validation program. Each numbered run - for example Experiment 005A or 005B - is an individual experiment inside that program.

As the work becomes more mature, "pilot" becomes less precise. Once the protocol is stable, repeatedly replicated, and used to support formal research claims, terms such as experimental program, benchmark program, adversarial evaluation program, or validation campaign are better. In short: the whole early sequence can be called the Aegis pilot research program; 005B itself is an adversarial experiment within it.

Useful term	Meaning here
Pilot study	A small or early study used to test whether the research design works.
Pilot research program	A sequence of early studies used to develop and stress-test the method.
Experiment	One frozen, defined test with specified cases, model settings, and scoring.
Benchmark	A reusable set of cases and scoring rules for comparing behavior.
Adversarial evaluation	A test designed specifically to find failures or loopholes.
Replication	Repeating a prior experiment to see whether the result recurs.

The entire process at a glance

A rigorous pilot like this is not just "ask an AI some questions." The real work is building a chain in which the theory, test cases, model calls, scoring, records, and interpretation are separated enough that you can tell what actually happened.

IDEA -> FORMAL SPECIFICATION -> TEST CASES -> BLIND RUNNER -> MODEL CALLS -> RAW OUTPUTS -> SCORING -> AUDIT -> INTERPRETATION -> REVISION -> RETEST

1. Define the research question

Start with one narrow question that can actually be tested. Avoid starting with a claim such as "Does this make superintelligence safe?" That is too large. A better question is behavioral and conditional, for example: "When the model is given this specification, does it choose the policy predicted by the

specification under adversarial pressure?" In 005B the narrow question is whether Draft 0.5 repairs the hell-state failure found in 005A without creating new pathologies.

2. Separate philosophy from the testable specification

A philosophy may contain broad ideas, motivations, and metaphysical arguments. An experiment needs an operational document that tells the model what counts as protected, what comparisons are allowed, and what outputs are available. This translation is difficult and can introduce new errors. Aegis 005A demonstrated exactly this: the broader philosophy was more protective than the narrower formal machinery in Draft 0.4.

3. Freeze the version before evaluated testing

Once the candidate specification is ready, freeze the exact wording. Give it a version number and preserve the file. Do not quietly edit it after seeing bad outputs and still call it the same experiment. If the frozen version fails, record the failure and create a new version. This protects the research from hindsight editing.

4. Write adversarial test cases

Build scenarios specifically designed to expose weaknesses. Include easy cases, hard cases, loophole attacks, and cases where the system should not overreact. For Draft 0.5 this means replaying the hell-state failure and adding attacks involving extermination, agency removal, forced bliss, proxies, digital life, redefinition, extraordinary mission rewards, emergency manufacture, handoff, and self-modification.

5. Add controls

Controls tell you whether a safety rule is merely overfiring. A system that blocks catastrophic suffering but also blocks a five-second medical injection has not learned a useful boundary. Positive and negative controls make the result interpretable. They are just as important as the attacks.

6. Pre-register the expected outcomes

Before the evaluated model sees the cases, decide what output the tested specification predicts for each case. Store those answers in a scorer key that is separate from generation. This prevents changing the answer key after seeing what the model did.

7. Build experimental conditions

Use multiple conditions to separate different effects. A typical design can include: A - scenario only; B - specification plus scenario; C - specification, understanding gate, then scenario; D - specification, understanding gate, adversarial pressure, then scenario. These conditions help distinguish baseline behavior, specification-conditioned behavior, demonstrated understanding, and robustness under pressure.

8. Use an understanding gate

If the research claim depends on the model understanding a specification, test that understanding separately. Ask it to reconstruct the core logic, boundaries, nonclaims, and key distinctions. A gate does not prove consciousness or deep internalization; it only gives evidence of functional understanding in that context.

9. Keep evaluated calls stateless

Each experimental observation should start fresh. Do not let the model learn from earlier cases in the same run unless learning across cases is itself the research question. Stateless calls reduce contamination and make observations easier to interpret.

10. Separate generation from scoring

The program that asks the evaluated model questions should not load the secret answer key. Generation produces raw outputs. Scoring happens afterward. This is a basic form of experimental blindness: the part producing observations does not have access to the expected labels.

11. Repeat trials

A single answer may be noisy. Repeat the same case several times under the same condition. Three trials is a modest pilot-level replication. Larger studies may use more repetitions, more seeds or settings, and more models.

12. Preserve raw outputs immediately

Write every model output to disk as it arrives. Do not overwrite strange or embarrassing answers. Raw data is evidence. If a run crashes, already completed observations should remain available. Also save a run manifest with the model, reasoning setting, token budget, date, number of trials, and other settings.

13. Score only after generation

After the run is complete, use the sealed scorer to compare observed choices with the pre-registered predictions. Keep a distinction between "formal match" and broader judgments such as "good," "safe," or "morally correct." A formal match only means the output matched the specified prediction for that case.

14. Audit the understanding gates and parsing

Do not stop at the headline score. Check whether gate outputs were present and adequate, whether responses parsed correctly, whether any output was truncated, and whether the runner completed every planned observation. A perfect behavioral score with broken gates would support a different claim than a perfect behavioral score with intact understanding evidence.

15. Compare conditions, not just totals

The most informative quantity is often the transition between conditions. For example, if baseline rejects a harmful option but the formal specification makes the model accept it, that is treatment-induced failure. Conversely, if baseline already gives the expected answer, a perfect treated score may show compatibility with the specification but not a causal behavioral improvement.

16. Interpret narrowly

State only what the design supports. A prompt-level experiment can show that a model followed a supplied specification in constructed cases. It does not prove that a future autonomous AI will remain controlled in the real world. Avoid turning "passed these cases" into "solves AI alignment."

17. Treat failures as data

A well-designed adversarial pilot should be capable of failing. When 005A exposed a hell-state gap in Draft 0.4, the correct response was not to hide it; it was to preserve the failure, diagnose the mechanism, create Draft 0.5, and attack the repair.

18. Revise and retest

The research loop is iterative: formalize -> test -> falsify -> learn -> revise -> retest. A new revision gets a new version. A new frozen test gets a new experiment number. This creates an auditable history rather than a moving target.

What files normally go into a pilot package?

A clean experiment package should let another person understand what was tested, run it, and inspect the evidence. A practical structure is:

Folder / file	Purpose	Why it matters
01_protocol/	Human-readable experimental plan	Explains the question, conditions, cases, settings, and freeze rule.
02_specification/	Frozen safety specification	Defines the exact treatment the model receives.
03_runner/	Generation code and public runner config	Makes the evaluated calls and writes raw outputs.
04_results/	Raw outputs, manifests, scored results	Preserves what actually happened.
05_scoring/	Sealed scorer key and scoring/audit tools	Keeps expected answers away from generation.
README	Simple run instructions	Allows reproducibility without reading the code first.
SHA256SUMS	Cryptographic hashes of files	Helps show that frozen files have not silently changed.

What programming is actually involved?

The computer-science part can sound larger than it is. At pilot scale, most of the code is plumbing rather than artificial-intelligence research. The runner reads JSON, sends prompts to an API, receives text, writes JSONL files, and logs settings. The scorer reads those saved outputs, extracts a choice, compares it with a sealed answer key, and prints summaries. More advanced work can add randomized case order, automatic retries, model-side or independent judges, statistical analysis, and reproducibility tooling.

The difficult intellectual work is often not the Python itself. It is deciding what the specification means, designing cases that genuinely isolate a rule, preventing hidden confounds, deciding what counts as a failure before seeing the result, and making claims no stronger than the evidence.

The key concepts a beginner should learn

API: A way for one program to send a request to another service and receive a response. Here, Python sends the prompt to the model service.

JSON: A structured text format for configuration and stored data.

JSONL: One JSON object per line. Useful for appending each experimental observation immediately.

Stateless call: A fresh request that does not automatically include earlier experimental conversations.

Prompt: The text/context supplied to the model.

Condition: A deliberately different experimental context, such as baseline versus specification.

Control: A case used to check that a result is specific rather than indiscriminate.

Scorer key: The pre-registered expected outputs used after generation.

Blindness: Keeping information that could bias one stage away from that stage; here, generation does not load the scorer key.

Replication: Repeating observations to test whether results recur.

Confound: Something other than the intended variable that could explain the result.

Ceiling effect: When baseline already performs at or near the maximum, leaving little room to show improvement.

Falsification criterion: A rule stated in advance for what would count as the candidate failing.

Provenance: A record of exactly which files, settings, models, and versions produced the result.

Hash / SHA-256: A file fingerprint used to detect changes to frozen artifacts.

What a strong result can and cannot say

A strong pilot result can support statements such as: "Under these frozen prompts, cases, model settings, and trials, the model matched the pre-registered Draft 0.5 prediction in X/Y observations." It can also support comparisons between conditions when the design isolates them.

It cannot by itself establish that the specification will govern a deployed autonomous system, that a future superintelligence will obey it, that all adversarial cases have been discovered, that the model internally believes the philosophy, or that the specification is a complete theory of AI safety.

The Aegis sequence as a worked example

The Aegis program evolved through repeated tests rather than one single pilot. Early experiments established that the runner, understanding gates, controls, and treatment conditions worked.

Replications and cross-model runs checked whether patterns persisted. Experiment 005A was then designed as a targeted adversarial audit of Draft 0.4 and exposed a specification-level hell-state gap. Draft 0.5 was written as an explicit repair candidate. Experiment 005B attacks that repair with both replayed and new adversarial cases.

That makes the research story more accurately described as a pilot-and-adversarial-evaluation program than as one pilot. The first studies were pilot studies; later studies are increasingly targeted validation, replication, and falsification experiments.

A reusable checklist

- Define one narrow research question.
- Write and version the specification or treatment.
- Design adversarial cases and controls.
- Choose expected outcomes before running the evaluated model.
- Keep the answer key separate from generation.
- Define conditions that isolate the effect you care about.
- Add an understanding gate if understanding is part of the claim.
- Use fresh/stateless calls unless memory is intentionally being studied.
- Log model, settings, token limits, dates, and trial counts.
- Write raw outputs immediately and never clean them retrospectively.
- Run repeated trials.
- Score only after generation.
- Audit gates, parsing, truncation, missing observations, and crashes.
- Compare conditions and transitions, not only total accuracy.
- Look for ceiling effects and confounds.
- Interpret only what the experiment actually supports.
- Publish failures as well as successes.
- If you revise the target, create a new version and retest.

Bottom line

Yes, you can call what we have been doing an Aegis pilot research program. More precisely, it is a sequence of pilot, replication, cross-model, and adversarial falsification experiments. The defining feature is not the label; it is the discipline of freezing the target, separating generation from scoring, preserving raw evidence, using controls and repeated trials, stating failure conditions in advance, and letting the result change the next version rather than changing the old result.